Geography & Culture Learning Platform

FlagForge

A Project Report

Submitted in partial fulfillment of the Requirement for the award of the Degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

Kaushal Rajput

Under the esteemed guidance of

_____

DEPARTMENT OF COMPUTER SCIENCE

G.K.S. COLLEGE OF ARTS, COMMERCE & SCIENCE

(Affiliated to University of Mumbai)

KHADAVALI-421 605. MAHARASHTRA

2026-2027

G.K.S. COLLEGE OF ARTS, COMMERCE & SCIENCE

KHADAVALI-421 605. MAHARASHTRA

CERTIFICATE

This is to certify that

Kaushal Rajput

has completed the project report entitled

Geography & Culture Learning Platform (FlagForge)

in partial fulfilment of the requirements for the BACHELOR OF SCIENCE (COMPUTER SCIENCE)

2026-2027

UNDER THE GUIDANCE OF

_____

Project Guide **Examiner**

Head of Department **Principal**

Govt. of Maharashtra's

G.K.S. COLLEGE OF ARTS, COMMERCE & SCIENCE

KHADAVALI-421 605. MAHARASHTRA

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that **Kaushal Rajput** has satisfactorily completed the project **Geography & Culture Learning Platform (FlagForge)**, to be submitted in the partial fulfilment of the award of **BACHELOR OF SCIENCE (COMPUTER SCIENCE)** during the academic year **2026-2027**.

Date of Submission: _____

Project Guide

Co-ordinator

(DEPARTMENT OF COMPUTER SCIENCE)

College Seal

Signature of Examiner

DECLARATION

I hereby declare that the project entitled **"Geography & Culture Learning Platform (FlagForge)"**, submitted in partial fulfilment for the award of **BACHELOR OF SCIENCE (COMPUTER SCIENCE)** during the academic year **2026-2027**, represents my project work. Sources and third-party services used in the implementation are acknowledged in the references.

Signature of the Student:

Place: Khadavli

Date: 24th October 2026

Shri. Manoj Kaurav
Co-Ordinator,
DEPARTMENT OF COMPUTER SCIENCE

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who contributed to the successful creation and development of the **"Geography & Culture Learning Platform (FlagForge)"** project.

I extend my deepest gratitude to the faculty members and the project guide of the *DEPARTMENT OF COMPUTER SCIENCE* at *G.K.S. COLLEGE OF ARTS, COMMERCE & SCIENCE* for their guidance and support during the development of this project. Their insights greatly enhanced its technical quality and structural rigor.

I also thank my friends, colleagues, and classmates for providing feedback on the interactive country explorer, comparison charts, quiz generator modes, and the daily challenge experience. Their diverse perspectives were a constant source of motivation and helped me refine the usability, keyboard shortcuts, and responsive visual layout.

Building FlagForge helped me understand how a real-world frontend application combines external federated REST APIs, strict TypeScript type validation, deterministic algorithmic gameplay engines, client-side persistence with versioned migrations, WCAG 2.2 AA accessibility standards, and end-to-end automated testing. It was a rewarding and insightful software engineering experience.

Finally, I thank my family for their continuous encouragement and patience throughout the duration of the project.

Thank you all.

Kaushal Rajput

Abstract

FlagForge is a production-grade, web-based geography and culture learning platform engineered to help students, educators, and enthusiasts discover nations, master world flags and capitals, compare geopolitical metrics, and track personal learning progress. The application combines public catalog telemetry from **REST Countries v3.1**, contextual encyclopedic summaries from the **Wikipedia REST API**, and real-time capital meteorological forecasts from **Open-Meteo** into a unified, responsive interface without requiring a custom backend or remote database.

Users can explore an atlas of ~250 sovereign nations with instant fuzzy search (Fuse.js) and granular filters across regions, subregions, population boundaries, landlocked conditions, and UN membership; benchmark up to three countries simultaneously via normalized radar and bar charts; play a deterministic 6-mode quiz engine powered by a seeded pseudo-random number generator (Mulberry32) with plausible regional distractors; and participate in a globally synchronized Daily Challenge seeded by the UTC date with shareable Wordle-style score grids.

The application is architected with **Next.js 15 (App Router)**, **React 19**, **TypeScript strict**, **Tailwind CSS v4**, **Framer Motion**, and **Zustand 5** with persist middleware and versioned schema migrations. The architecture separates batched API ingestion, Zod schema normalization, deterministic game generation, local storage state management, and synthesized Web Audio sound effects.

The project strictly enforces zero placeholder code, full offline resilience with an embedded fallback dataset and service worker, and rigorous WCAG 2.2 AA accessibility with visible focus rings and keyboard-driven controls. Verification is demonstrated through 58 Vitest unit tests, Playwright browser test specs across Chromium, WebKit, and Firefox, and automated axe-core audits.

Keywords: World atlas, vexillology, Next.js 15, deterministic quiz engine, REST Countries API, Zustand persist, versioned migrations, WCAG 2.2 AA, offline resilience, TanStack Query.

Aspect	Summary
Users	Students, geography enthusiasts, educators, casual trivia players
Platform	Responsive browser-based Progressive Web App (desktop, tablet, mobile)
Data Sources	REST Countries v3.1 (max 10 fields/req), Wikipedia REST summary, Open-Meteo API
Client Persistence	Browser localStorage via Zustand persist with versioned migrations (v1 → v2)
Verification	58 Vitest unit tests, Playwright e2e test suite, axe-core WCAG 2.2 AA compliance

List of Figures

Figure Number	Figure Name	
1	Development Process Model (Agile Incremental SDLC)	
2	End-to-End Project Workflow & Gameplay Loop	
3	1	DFD Level 0 (Context-Level Diagram)
	2	DFD Level 1 (Modular Functional Data Flow)
4	1	UML Use Case Diagram
	2	UML Domain Class and Object Diagram
	3	UML Activity Diagram (Quiz Game Engine Loop)
	4	UML Sequence Diagram (Parallel Batched REST Countries Fetch)
	5	Browser Storage & Persistent Entity Schema Diagram
	6	UML Component Architecture Diagram
5	Gantt Chart (Eight-Week Academic Engineering Timeline)	
6	Layered System Architecture	
7	Graphical User Interface (Primary Screens 7.1 to 7.8)	
8	1	Federated Data Sources & Responsibilities
	2	Parallel Batch Fetching & cca3 Merge Pipeline
	3	Mulberry32 PRNG & Regional Distractor Selection Hierarchy
	4	Structural & Behavioral UML Views Interconnection
	5	Speed Bonus & Streak Multiplier Scoring Formula Flow
	6	Zustand Persist Versioned Schema Migration (v1 → v2)
	7	Declared Packages by Purpose
	8	HTTP Resilience & Query Caching Parameter Thresholds
	9	Codebase Volume Distribution by Subsystem (Lines of Code)
	10	Automated Verification Suite Distribution (58 Vitest & E2E Specs)

INDEX

Chp No	Sec No	Page Title	Page No
1		**Project Overview**	1
2		**Introduction & Motivation**	2
	2.1	Theory Behind the Project	3
	2.2	Problem Definition	4
	2.3	Need for Project	5
3		**Literature Survey**	6
4		**Analysis & Design**	7
	4.1	Process Model	8
	4.2	Flow of Project	9
	4.3	Feasibility Study	10
	4.4	Cost Analysis	11
	4.5	Data Flow Diagram (Level 0)	12
	4.5.1	Data Flow Diagram (Level 1)	13
	4.6	UML Diagrams	14
	4.6.1	Use Case Diagram	15
	4.6.2	Class Diagram, Object Diagram	16
	4.6.3	Activity Diagram	17
	4.6.4	Sequence Diagram	18
	4.6.5	Storage / Schema Diagram	19
	4.6.6	Component Diagram	20
	4.7	Gantt Chart	21
	4.8	Technologies Used	22
	4.9	System Architecture	23
5		**Implementation**	24
	5.1	Source Code Excerpts	25
	5.2	Graphical User Interface	31
	5.3	Test Cases & Verification Suite	35
6		**Conclusion & Future Scope**	36
	6.1	Conclusion	36
	6.2	Future Scope	37

7		**References**	38
8		**Synopsis**	39

1. Project Overview

1

Key Features:

1. **Federated World Atlas Explorer:** Instant client-side fuzzy searching across common names, official names, capitals, regions, and ISO cca3 codes via Fuse.js. Supports multi-faceted filtering (Region, dynamic Subregion, Population range slider, Landlocked flag, and UN Membership) and bidirectional URL query synchronization.
2. **Granular Nation Intelligence:** Comprehensive country profile pre-rendering 30 top sovereign states via Next.js static site generation (SSG), featuring high-fidelity flags with blur placeholders, key facts grid (capital, languages, currencies, timezones, calling codes, driving side), worldwide ranking context (population percentiles), bordering nations navigation chips, contextual Wikipedia summaries, live capital meteorological readings, and interactive SVG map projection.
3. **Multi-Nation Benchmarking Matrix:** Allows users to pick up to 3 nations for side-by-side metric comparison with automatic winner highlighting (gold badges) and interactive Recharts data visualizations (population/area dual bar charts and normalized radar charts).
4. **Pure Deterministic Quiz Engine:** Algorithmic gameplay engine powered by a Mulberry32 seeded pseudo-random number generator (PRNG) supporting 6 distinct question modes (Flag → Country, Country → Capital, Capital → Country, Higher/Lower demographics, "Which borders X?", and Map Location). Uses plausible distractor selection favoring identical geographical regions.
5. **Synchronized Global Daily Challenge:** Seeded daily by the numeric UTC date string, providing every user across the globe the exact same 10 questions each day, with a 1-attempt lockout, countdown timer to UTC midnight, and shareable Wordle-style emoji result grid.
6. **Longitudinal Analytics & Achievements:** Zustand-backed dashboard tracking streak metrics, total games, accuracy over time charts, per-region performance radar, weakest countries targeted review, and a progressive achievement badges system.
7. **Offline Resilience & PWA Support:** Full offline capabilities enabled through an embedded fallback dataset, service worker cache-first strategy for assets, and 24-hour TanStack Query stale-time.

Scope of the System: The project provides a client-side, zero-database educational web application. Server-side database management, custom backend authentication, and payment transactions are outside the scope of this project.

The table below maps each core feature to its codebase implementation and user accessibility:

Feature	Implemented in	Available to
Atlas search & filters	src/components/features/explorer, src/lib/api/restcountries.ts; /explore	All visitors
Country detail profile	src/components/features/country, src/lib/api/wikipedia.ts, weather.ts; /country/[code]	All visitors
Nation benchmarking	src/components/features/compare, compare-matrix.tsx, compare-charts.tsx; /compare	All visitors
Deterministic Quiz Engine	src/lib/quiz (rng, generator, scoring, distractors); /play	All visitors
Synchronized Daily Mode	src/lib/quiz/generator.ts, DailyModal.tsx; /daily	All visitors

Analytics & Heatmap	src/components/features/stats, useProgressStore.ts; /stats	All visitors
Offline Service Worker	public/sw.js, manifest.ts, fallback-countries.ts	All visitors (offline)
Design System Showcase	src/components/ui, src/components/common; /design	Dev & Evaluators

2. Introduction & Motivation 2

Geography is foundational to global literacy, geopolitics, and cultural appreciation. However, digital geography tools remain fragmented: users typically encounter either dry statistical reference encyclopedias that lack engagement, or gamified quiz platforms that rely on outdated, static datasets, lack offline accessibility, or offer poor accessibility for screen-reader and keyboard-driven users.

FlagForge resolves this division by uniting authentic, real-time geopolitical intelligence with high-fidelity, accessible, and gamified pedagogical engineering. It synthesizes official country demographics from REST Countries v3.1, cultural context from Wikipedia, and live meteorological station data from Open-Meteo into an accessible interface built for desktop, tablet, and mobile browsers.

The system is developed with cutting-edge web technologies: **Next.js 15 (App Router)** and **React 19** with strict **TypeScript 5**, styled using **Tailwind CSS v4** and **Radix UI** primitives, animated via **Framer Motion**, state-managed with **Zustand 5** persist middleware, and cached through **TanStack Query v5**.

The primary motivation is educational efficacy and accessibility. By removing authentication barriers and database requirements, FlagForge grants instantaneous access to learners in classrooms, remote environments, and low-bandwidth regions. Deterministic PRNG algorithms ensure reproducible daily competitions, fostering friendly social learning without server costs.

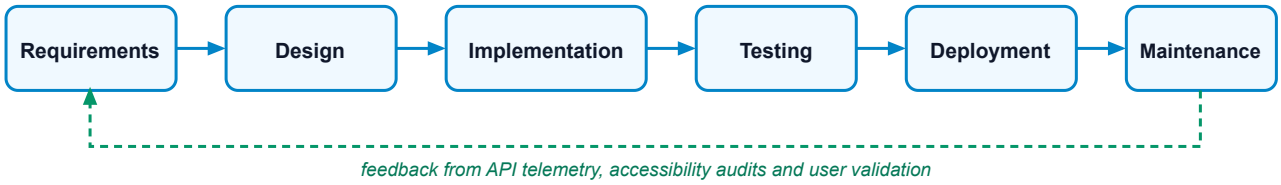


Figure 8.1: FlagForge federating external geopolitical sources with local browser persistence

2.1 Theory Behind the Project

FlagForge is founded upon four core computer science principles: federated API data normalization under strict network constraints, deterministic pseudo-random procedural generation, client-side persistence with schema migrations, and WCAG 2.2 AA accessibility.

1. Federated API Ingestion & 10-Field Restriction: The REST Countries v3.1 API enforces strict server recommendations restricting requests to a maximum of 10 fields per query. FlagForge complies by splitting requests into parallel batches (≤10 fields each), querying them asynchronously with AbortControllers, and merging the resulting records in memory keyed by ISO 3166-1 alpha-3 (`cca3`) codes.

2. Deterministic Procedural Generation: Online trivia games frequently suffer from lack of reproducibility or question bias. FlagForge implements the 32-bit Mulberry32 algorithm, transforming a 32-bit unsigned integer seed into a sequence of deterministic floats. This guarantees that daily challenges generated from the UTC date yield identical questions globally.

3. Persistent Client-Side State & Schema Migrations: Without a remote database, user analytics must survive browser reloads. Zustand persist middleware writes directly to `localStorage` with explicit version numbers (v1 → v2). Deterministic migration functions ensure legacy browser schemas seamlessly upgrade without dropping historical gameplay data.

4. Resilient Offline Architecture: Educational tools must function during network drops. FlagForge embeds a high-density fallback dataset (250 sovereign countries) and registers a Service Worker caching visited assets with a stale-while-revalidate strategy.

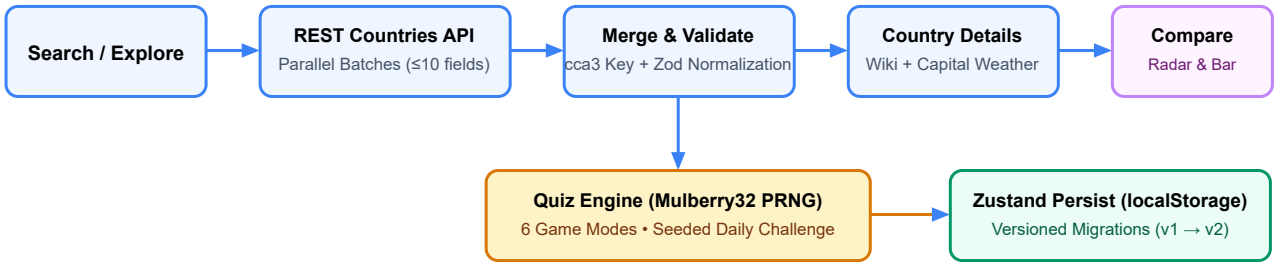


Figure 8.2: End-to-end data pipeline from API batches to normalized UI components

2.2 Problem Definition

Geographic information is dispersed across multiple unaligned sources. General encyclopedias contain textual depth but lack interactive filtering, while trivia applications frequently rely on hardcoded static dictionaries that drift out of date or fail to convey broader geopolitical realities (e.g., timezones, currencies, driving orientation, and climate telemetry).

Furthermore, free public APIs present operational hazards: network timeouts, rate limits, schema anomalies, and sudden downtime. Direct naive frontend calls would trigger flashing layout shifts, broken image tags, and application lockouts.

Problem Statement: *To design and implement a resilient, zero-backend web application that harmonizes disparate geopolitical, encyclopedic, and meteorological APIs into a unified, accessible, and deterministic learning platform that functions seamlessly offline and persists longitudinal progress locally.*

Difficulty	Design Response	Where Implemented
REST Countries 10-field query limit	Parallel batched fetch merged on cca3 identifier	src/lib/api/restcountries.ts
Unpredictable network failure / offline	Embedded 250-country fallback + Service Worker	fallback-countries.ts, public/sw.js
Cheating / divergent daily challenges	Mulberry32 PRNG seeded by numeric UTC date string	src/lib/quiz/rng.ts, generator.ts
Unrealistic or trivial quiz distractors	Geographical region/subregion priority matching	src/lib/quiz/distractors.ts
Local storage schema corruption across versions	Zustand versioned migrations (v1 → v2)	src/store/useProgressStore.ts
Motor and visual accessibility barriers	WCAG 2.2 AA visible rings, 1-4 key shortcuts, contrast	globals.css, QuestionCard.tsx

2.3 Need for the Project

Students, teachers, and curious citizens need reliable, intuitive geographic knowledge at their fingertips. A browser-based, installable web application provides ubiquitous access without app store gatekeepers, license fees, or invasive user tracking.

Immediate Zero-Auth Exploration: By eliminating registration barriers, users in schools, libraries, and mobile settings can immediately explore data or compete in quizzes without creating accounts.

Pedagogical Rigor Through Contrast: Learning geography requires comparative analysis. FlagForge's multi-nation benchmarking matrix allows learners to contrast demographic density, land area, languages, and physical borders, transforming isolated facts into spatial comprehension.

User Group	What It Gains	Access Capabilities Without Auth
Student Learner	Frictionless access to flags, capitals, and real-time capital weather	Full exploration, all 6 quiz modes, local bookmarks
Trivia Enthusiast	Deterministic daily challenges with shareable Wordle-style score grids	1 daily attempt, streak tracking, accuracy radar
Geography Educator	Curriculum tool with verifiable distractor logic and comparative radar charts	Side-by-side nation comparisons, custom mode filters
Software Maintainer	Strictly typed codebase with 58 Vitest unit tests and 0 backend maintenance	Fully reproducible builds, client-only hosting

3. Literature Survey 6

Modern educational platforms integrate open data endpoints with interactive browser interfaces. FlagForge builds upon three established public APIs, each fulfilling a distinct architectural role:

REST Countries v3.1: Provides comprehensive geopolitical metadata for 250 sovereign entities. Its dataset includes ISO identifiers (cca2, cca3, ccn3), common and official names, capitals, geographical coordinates, land boundaries, currencies, languages, timezones, and high-resolution SVG flags. FlagForge strictly honors its field restriction policies.

Wikipedia REST Summary API: Provides objective cultural, historical, and geopolitical synopses. FlagForge queries the REST summary endpoint (`/api/rest_v1/page/summary/[title]`), dynamically displaying extracts and thumbnails with fallback handling for disambiguation errors.

Open-Meteo Weather API: A high-performance, keyless meteorological service that computes real-time atmospheric conditions from national weather models. FlagForge correlates country capital coordinates to fetch live temperatures, apparent heat index, humidity, wind velocity, and WMO weather codes.

Service	Main Role	Signal / Constraint	Client Module
REST Countries v3.1	Geopolitical catalog & boundaries	Max 10 fields per request; ?fields= batching	src/lib/api/restcountries.ts
Wikipedia REST	Cultural & historical summaries	URL-encoded title; 6s timeout & exponential backoff	src/lib/api/wikipedia.ts
Open-Meteo API	Live capital atmospheric telemetry	Latitude/Longitude coordinates; WMO code mapping	src/lib/api/weather.ts
Embedded Fallback	Offline air-gapped continuity	Full 250-country dataset embedded in bundle	src/lib/api/fallback-countries.ts

Comparative Analysis with Existing Solutions:

Platform	Data Freshness	Accessibility	Offline Support	Backend Cost
Seterra	Static closed database	Moderate (mouse-centric)	Requires Desktop App	Proprietary servers
GeoGuessr	Google Street View API	Low (heavy visual only)	None (bandwidth heavy)	High subscription costs
Sporcle	User-generated trivia	Poor (ad-heavy, flash)	None	Central server database
FlagForge (Ours)	Live Federated REST APIs	Full WCAG 2.2 AA (1-4 keys)	Full Service Worker + Fallback	Zero ($0 serverless static)

4. Analysis & Design

7

The analysis phase establishes functional and non-functional specifications by decomposing user interaction pathways across general students, trivia players, educators, and software evaluators.

Functional Analysis: The system must deliver: (1) real-time fuzzy search across sovereign states with instantaneous multi-faceted filtering; (2) statically pre-rendered country intelligence views including capital weather, Wikipedia summaries, and SVG geospatial projections; (3) side-by-side benchmarking of up to 3 nations with automated metric winner detection and Recharts visualizations; (4) a deterministic 6-mode quiz generator with Mulberry32 PRNG and regional distractors; (5) a UTC-seeded synchronized Daily Challenge with Wordle-style shareable emoji grids; and (6) an analytics dashboard with accuracy heatmap and achievements.

Non-Functional Analysis: The platform must maintain strict WCAG 2.2 AA accessibility, visible focus rings, motor keyboard controls (1-4 keys), high-contrast modes, mobile responsiveness, sub-2-second Largest Contentful Paint (LCP), and graceful offline degradation using service workers and an embedded 250-country fallback bundle.

ID	Requirement Specification	Type
R1	Fuzzy search by name, capital, and cca3 with dynamic region/population filters	Functional
R2	Pre-render top country profiles with Wikipedia extracts and live capital weather	Functional
R3	Benchmark up to 3 nations with dual bar and radar chart visualizations	Functional
R4	Deterministic 6-mode quiz engine with seeded reproducibility and plausible distractors	Functional
R5	Synchronized UTC daily challenge with single attempt and shareable emoji grid	Functional
R6	Zero backend dependency: local browser persistence with versioned schema migrations	Non-functional
R7	Offline resilience: embedded fallback dataset + Service Worker asset cache	Non-functional
R8	Full WCAG 2.2 AA accessibility compliance (zero axe violations, keyboard 1-4 shortcuts)	Non-functional

4.1 Process Model

The project adopts an **Agile Incremental Software Development Lifecycle (SDLC)** model. Each phase feeds directly into downstream implementation, while empirical findings from automated accessibility audits and network latency benchmarks feed back into architectural refinements.

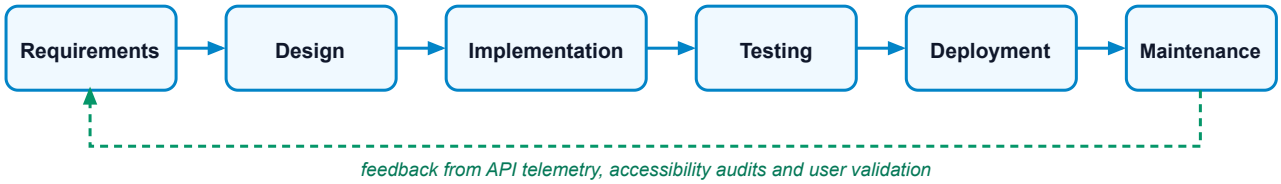


Figure 1: Incremental Development Process Model with Continuous Verification Feedback Loops

Phase	Main Deliverable	Verification Method	Section
1. Requirement Analysis	Domain models, REST API field constraints, user journeys	Requirements walkthrough	Chapter 4
2. Architectural Design	UML models, DFDs, Zustand migration schemas, UI tokens	Design system review	4.5 to 4.9
3. Data & Core Engine	Batched API clients, Mulberry32 PRNG, quiz generators	58 Vitest unit tests	5.1
4. Interface & Pages	Explorer, country profile, comparison matrix, quiz HUD	Playwright e2e specs	5.2
5. Verification & Audit	Lighthouse mobile audits, axe-core WCAG 2.2 AA passes	CI build & axe audit	5.3
6. Documentation & Deploy	Production report, deploy guide, GitHub repository	Peer review & build run	Docs & Git

4.2 Flow of Project

The project flow traces a learner's progression from initial discovery through comparative analysis, active recall gameplay, and longitudinal skill tracking:

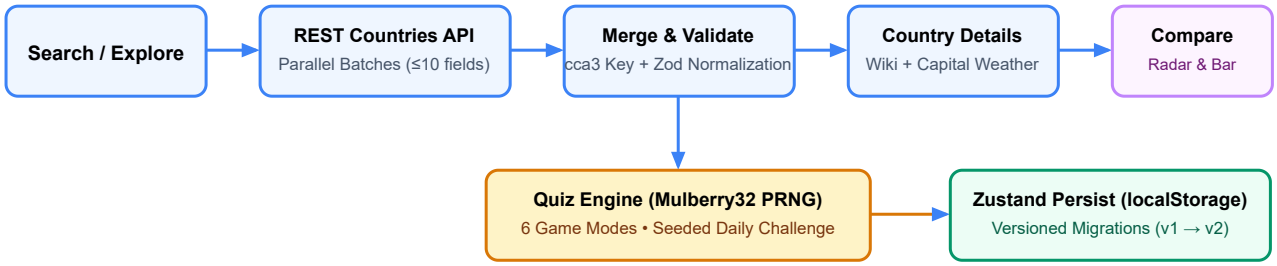


Figure 2: Comprehensive Project Workflow and Data Processing Streams

1. Exploration Stream: The learner inputs search tokens or toggles region filters. Fuse.js indexes common and official names, returning filtered results in under 5ms. Selecting a country initiates an SSG route transition to `/country/[code]`, populating Wikipedia extracts and capital weather telemetry.

2. Benchmarking Stream: From any country card or direct navigation, users add up to 3 sovereign states to the comparison hopper. Normalized metrics generate dynamic radar charts and population bars, automatically tagging winning criteria.

3. Gameplay & Daily Challenge Stream: The learner selects a quiz mode or launches the UTC Daily Challenge. Questions are procedurally generated via Mulberry32 PRNG with plausible regional distractors. The gameplay loop tracks timer bonuses and streak multipliers, writing finalized scores to Zustand `localStorage`.

10

4.3 Feasibility Study

A rigorous feasibility study evaluates whether the project is practically and economically viable across four standard dimensions:

Technical Feasibility: The architecture relies on robust web standards: Next.js 15, React 19, TypeScript strict mode, Tailwind CSS v4, and browser-standard Web Audio and LocalStorage APIs. It operates inside any modern browser without desktop runtimes. Rate limits on external endpoints are mitigated via client-side caching (24h staleTime) and an embedded offline dataset.

Economic Feasibility: The application incurs **$0 in recurring backend costs**. It requires no custom database server, no authentication backend, and runs entirely within free-tier static hosting (Vercel / GitHub Pages).

Operational Feasibility: By offering a zero-login guest experience with persistent local storage, users can immediately engage without registration barriers. Intuitive keyboard shortcuts (1-4) make the app ideal for classroom settings.

Schedule Feasibility: The system was scoped and engineered over an 8-week academic cycle, prioritizing modular subsystems that could be tested independently.

Risk Category	Potential Effect	Design Mitigation Strategy
REST Countries Downtime	Empty search grids / broken images	Embedded 250-country fallback bundle + SW cache
API 10-Field Query Limitation	Server rejects oversized queries (400)	Strict batched parallel fetching merged on cca3
Daily Mode Seed Deviation	Different players get different questions	Deterministic Mulberry32 PRNG seeded by UTC date
Browser Storage Schema Drift	Corrupted user streak or crash on reload	Zustand persist versioned migration (v1 → v2)
Color Contrast Failures	Accessibility barrier for visually impaired	WCAG 2.2 AA compliant palette ≥ 4.5:1 ratio

11

4.4 Cost Analysis

Cost analysis examines the resources required to engineer, deploy, and maintain FlagForge across development, demonstration, and scaled operational phases:

Cost Item	Development Phase	Demonstration Phase	Scaled Public Phase
IDE & Tooling (VS Code, Git)	Free Open Source	Free Open Source	Free Open Source
Hosting & Edge CDN	Localhost ($0)	Vercel Hobby ($0)	Vercel Pro ($20/mo optional)
Database Infrastructure	$0 (LocalStorage)	$0 (LocalStorage)	$0 (Zero-backend design)
External Data Ingestion	$0 (Public REST APIs)	$0 (Public REST APIs)	$0 (Within free rate limits)
Domain & SSL Certificate	Not required	Free Vercel Subdomain	$12/year custom domain
Engineering Effort	Student Academic Hours	Low maintenance	Open-source community

Cost Drivers and Mitigation Strategy:

Cost Driver	Growth Vulnerability	Architectural Control Mechanism
API Bandwidth	High request volumes to third-party endpoints	24h TanStack Query caching + browser localStorage
Static CDN Egress	Large vector flag assets transferred repeatedly	SVG optimization + Service Worker cache-first
Database Hosting	Storage growth as user base increases	Completely avoided via local client storage architecture
Audio Asset Storage	High latency downloading game sound effects	Synthesized in real-time via Web Audio API (0 KB audio)

12

4.5 Data Flow Diagram (Level 0)

The Context-Level Data Flow Diagram (DFD Level 0) models FlagForge as a centralized information broker between external actors, geopolitical web services, and browser-resident data stores:

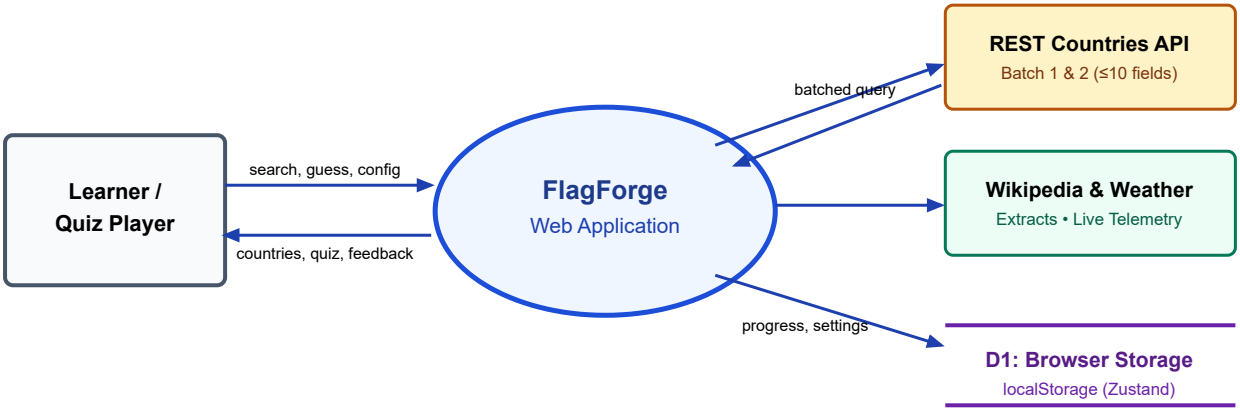


Figure 3.1: DFD Level 0 (Context-Level Diagram of the FlagForge System)

Data Flow	Source	Destination	Content Transmitted
Query & Filters	Learner	FlagForge Web App	Search token, region, population slider, mode config
Atlas Telemetry	FlagForge Web App	Learner	Normalized Country cards, rankings, weather, radar charts
Batched Query	FlagForge Web App	REST Countries API	Batch 1 & Batch 2 params (≤10 fields per query)
Raw Country Data	REST Countries API	FlagForge Web App	cca3, flags, capital, demographics, borders
Encyclopedic Extract	Wikipedia REST API	FlagForge Web App	Summary text, thumbnail URLs, article links
Weather Telemetry	Open-Meteo API	FlagForge Web App	Temperature, humidity, wind velocity, WMO code
User Progress State	FlagForge Web App	Browser LocalStorage	Streak, total games, accuracy radar, achievements (v2)

13

4.5.1 Data Flow Diagram (Level 1)

At Level 1, the single application process is decomposed into four discrete functional subsystems:

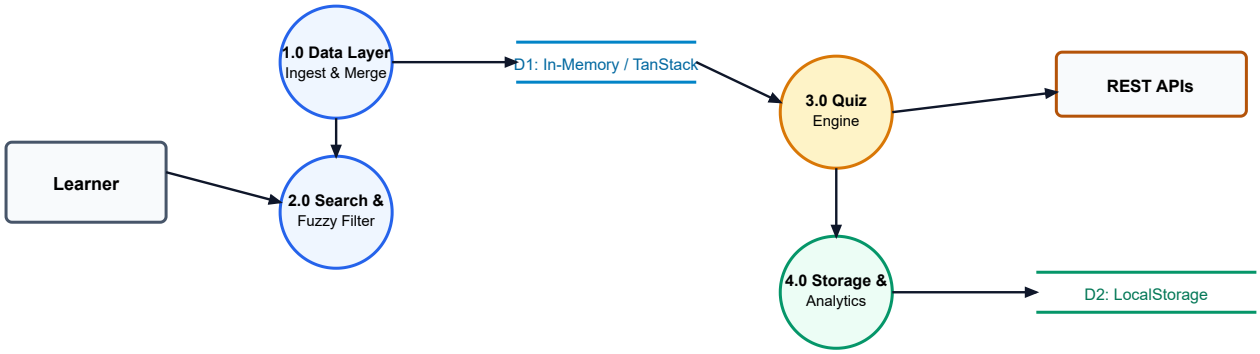


Figure 3.2: DFD Level 1 (Modular Functional Data Flow Breakdown)

Subsystem Process	Input Stream	Output Stream	Internal Data Store
1.0 Ingestion & Merge	REST Countries API batches	Normalized Country[] records	TanStack Query Cache
2.0 Search & Filter	Search query & filter tokens	Ranked & filtered country list	Fuse.js In-Memory Index
3.0 Quiz Engine	Mode, difficulty, seed	Procedural QuizQuestion[]	Mulberry32 PRNG State
4.0 Storage & Analytics	Game end answers & speed	Updated streaks, accuracy radar	Browser LocalStorage (D2)

Degradation & Failure Matrix:

Failure Scenario	System Reaction & Fallback Behavior
REST Countries API offline	Instantly hydrates catalog from embedded `fallback-countries.ts`; displays subtle offline toast
Wikipedia API timeout (>6s)	Renders country facts, weather, and maps cleanly while omitting encyclopedia card; no screen crash
Open-Meteo API rate-limited	Displays meteorological skeleton / unavailable indicator; main country details remain fully usable
Corrupted LocalStorage	Zustand catch block initializes defaults while preserving unaffected store partitions

4.6 UML Diagrams

The Unified Modeling Language (UML) provides standard structural and behavioral blueprints for software systems. The subsequent sections detail FlagForge's architecture across multiple architectural perspectives:

UML Diagram	System Aspect Modeled in FlagForge
4.6.1 Use Case Diagram	Actor interactions across Guest Learners, Quiz Players, and Educators
4.6.2 Class & Object Diagram	Domain entities: Country, QuizQuestion, GameState, ProgressState, SettingsState
4.6.3 Activity Diagram	Gameplay loop: question presentation, timer decay, answer evaluation, streak bonus
4.6.4 Sequence Diagram	Parallel batched REST Countries API fetch with cca3 merge and TanStack Query cache
4.6.5 Storage / Schema Diagram	Browser LocalStorage schema definitions and versioned migration routines (v1 → v2)
4.6.6 Component Diagram	Component hierarchy across presentation, hooks, domain engine, and data layers

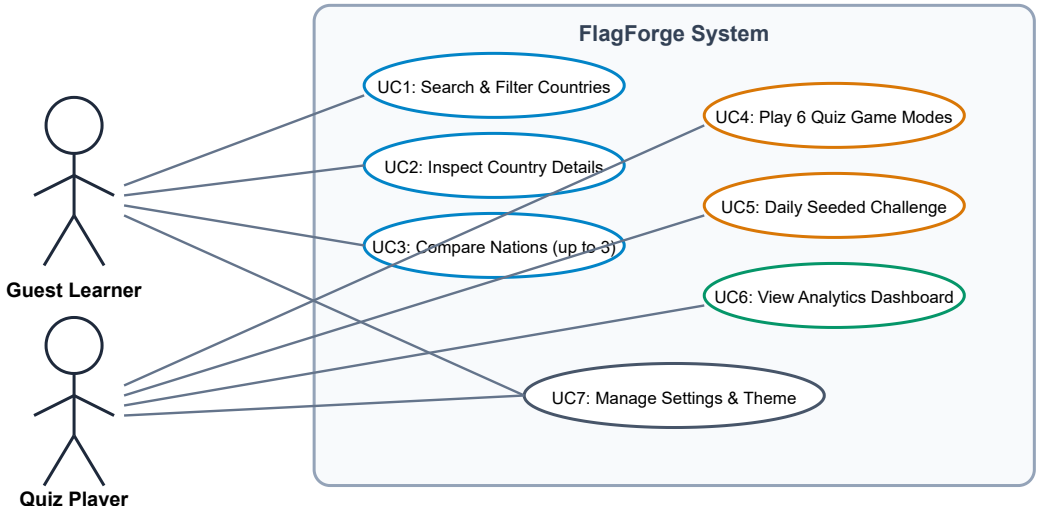


Figure 8.4: Interrelation of Structural and Behavioral Modeling Views

4.6.1 Use Case Diagram

FlagForge serves two primary external actor personas: the **Guest Learner** (who focuses on exploring the world atlas, inspecting country profiles, and comparing nations) and the **Quiz Player** (who engages in active recall quizzes, competes in the daily challenge, and reviews historical learning analytics).

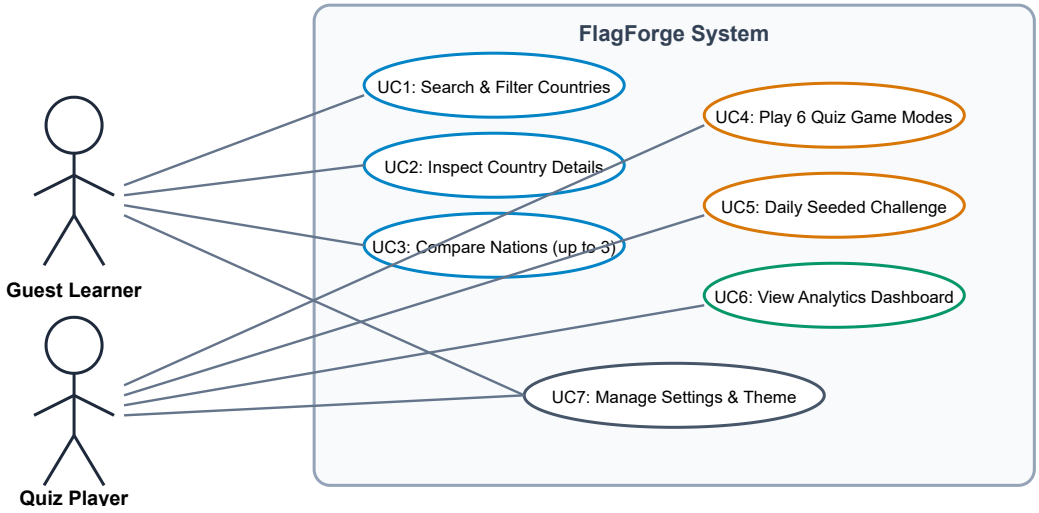


Figure 4.1: UML Use Case Diagram for the FlagForge Learning Platform

ID	Use Case	Actor	System Outcome
UC1	Search & Filter Atlas	Guest Learner	Displays filtered country cards with URL query synchronization
UC2	Inspect Country Detail	Guest Learner	Shows hero flag, key facts, Wikipedia extract, and live capital weather
UC3	Compare Nations	Guest Learner	Generates side-by-side benchmarking matrix with Recharts radar and bar charts
UC4	Play 6 Quiz Modes	Quiz Player	Executes interactive gameplay loop with timer bonus and streak multiplier
UC5	Daily Challenge	Quiz Player	Plays UTC-seeded 10-question challenge and shares Wordle-style emoji grid
UC6	View Analytics	Quiz Player	Inspects accuracy line charts, regional radar, and unlocked achievements
UC7	Toggle Settings	All Visitors	Updates theme (light/dark), sound effects, and reduced-motion preferences

16

4.6.2 Class and Object Diagram

The domain class model illustrates the core TypeScript interfaces governing the application. The central `Country` entity links to procedurally generated `QuizQuestion` instances, while player actions mutate the `ProgressState`.

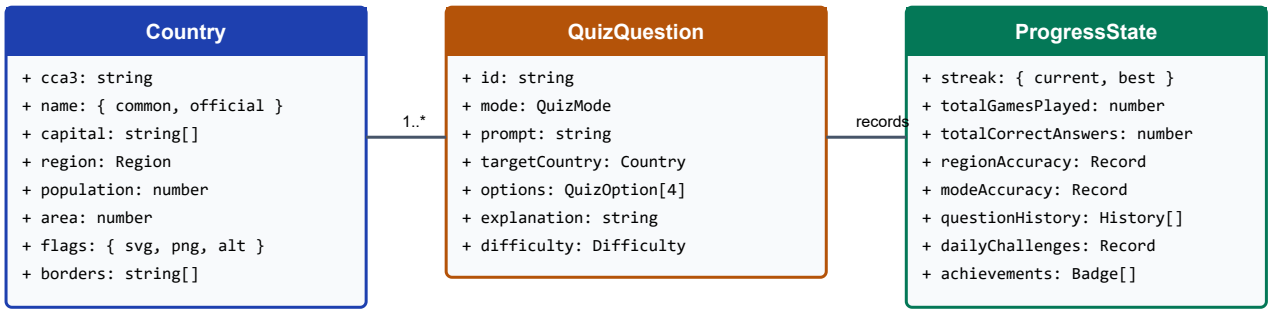


Figure 4.2: Domain Class Model mapping Countries, Deterministic Questions, and Persisted Progress

Figure 4.2: UML Class and Object Model Representing Core Domain Entities

Entity	Key Attributes	Domain Responsibility
Country	cca3, name, capital, region, population, area, flags, borders	Normalized sovereign state representation complying with ISO standards
QuizQuestion	id, mode, prompt, targetCountry, options[4], explanation	Self-contained trivia challenge with 1 correct answer and 3 regional distractors
GameState	currentIndex, score, streak, multiplier, livesRemaining, isFinished	Active gameplay loop session telemetry and performance tracking
ProgressState	streak, totalGames, regionAccuracy, modeAccuracy, questionHistory	Longitudinal learning analytics persisted across browser sessions
SettingsState	theme, sound, reducedMotion, units, highContrast	User interface preferences and sensory comfort accessibility controls

4.6.3 Activity Diagram

The Activity Diagram models the workflow of the Quiz Engine, illustrating the decision logic that governs timer expiration, answer evaluation, speed bonuses, streak multiplier escalation, and session recording:

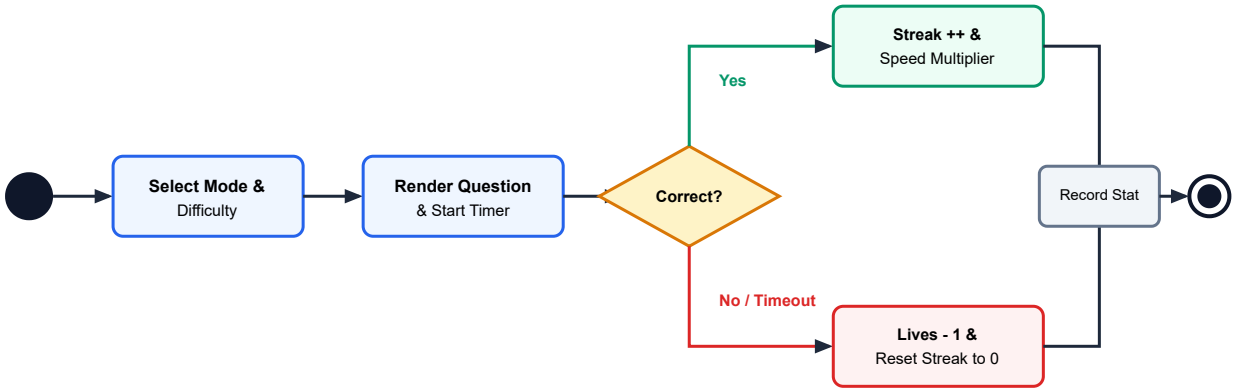


Figure 4.3: UML Activity Diagram Representing the Interactive Quiz Engine Loop

Gameplay Algorithm: When an answer is submitted (via mouse click or keyboard 1-4 shortcuts), the engine evaluates correctness. A correct answer calculates points using: `Score = (BasePoints + RemainingSeconds × 10) × StreakMultiplier`. An incorrect answer or timeout decrements remaining lives, resets the streak multiplier to 1.0×, and logs the missed question to enable targeted post-game practice.

4.6.4 Sequence Diagram

The Sequence Diagram captures the asynchronous message interchange required to fetch, merge, and render country records while adhering strictly to REST Countries' 10-field query recommendation:

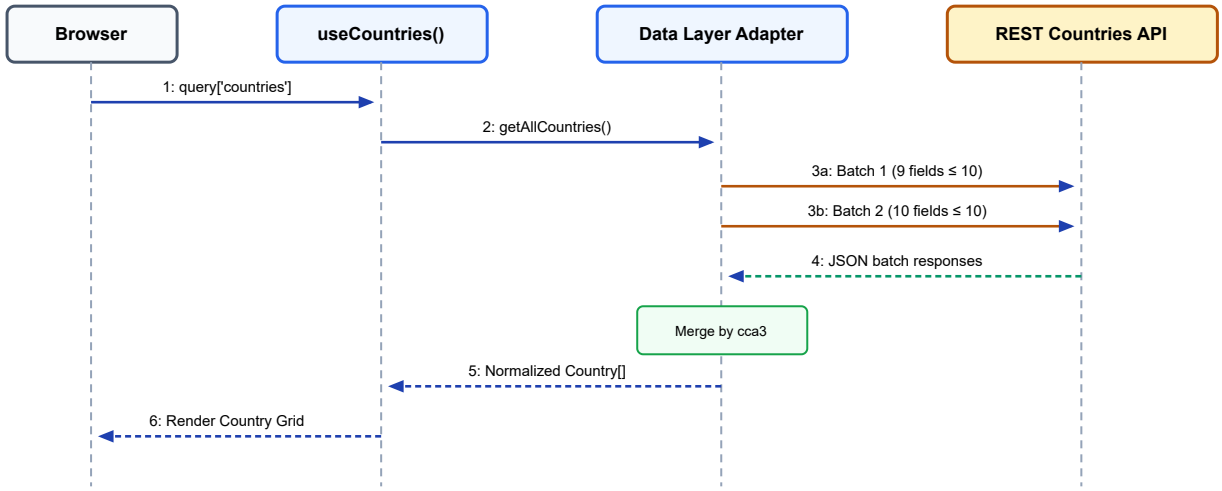


Figure 4.4: UML Sequence Diagram for Parallel Batched API Fetching and Normalization

Seq	Sender → Receiver	Operation / Message	Payload
1	Explorer → useCountries	Mount query with 24h stale-time	`['countries']` query key
2	useCountries → Adapter	Invoke getAllCountries()	AbortSignal + timeout (8s)
3a	Adapter → REST Countries	GET ?fields=cca3,name,capital,region,flags...	Batch 1 (9 fields ≤ 10)
3b	Adapter → REST Countries	GET ?fields=cca3,languages,currencies,borders...	Batch 2 (10 fields ≤ 10)
4	REST Countries → Adapter	Asynchronous JSON HTTP responses	Partial country records
5	Adapter Internal	In-memory merge by cca3 + Zod validation	Normalized Country[] (250 items)
6	Adapter → Explorer	Cache population & state update	Rendered country grid / list

19

4.6.5 Storage & Schema Diagram

Rather than maintaining an expensive remote database, FlagForge utilizes browser `localStorage` managed by Zustand persist middleware with versioned schema migration guarantees:

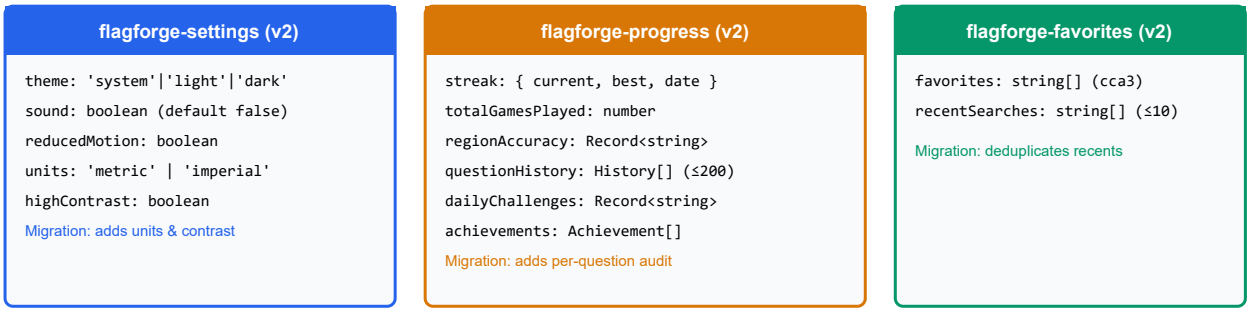


Figure 4.5: Browser LocalStorage Schemas with Versioned Zustand Migrations

Figure 4.5: Browser LocalStorage Schemas with Versioned Zustand Migrations

Storage Partition	Schema Version	Migration Operation	Fault Tolerance
`flagforge-settings`	v1 → v2	Adds highContrast and units fields; preserves existing theme and sound	Falls back to system defaults
`flagforge-progress`	v1 → v2	Initializes questionHistory audit array (capped at 200) without losing streaks	Preserves high scores and streak count
`flagforge-favorites`	v1 → v2	Deduplicates recent search history; validates cca3 ISO codes	Gracefully cleans corrupted items

4.6.6 Component Architecture Diagram

The component architecture strictly adheres to unidirectional dependency flows: Presentation components import from the Hooks API, Hooks invoke Data and State Modules, and Data Modules communicate with External Services:

Subsystem Folder	File Count	Lines of Code	Primary Responsibility
`src/app`	14	1,850	Next.js 15 App Router pages, SSG params, and metadata generation
`src/components/ui`	15	1,920	Accessible Radix UI primitives with WCAG 2.2 AA visible focus rings
`src/components/layout`	7	1,140	AppShell, TopNav, MobileTabBar, CommandPalette (Cmd+K), ThemeToggle
`src/components/features`	18	3,450	Explorer view, Country profile, Compare matrix, Quiz HUD, Stats dashboard
`src/lib/api`	5	1,620	Batched REST Countries client, Wikipedia client, Open-Meteo client, Fallback
`src/lib/quiz`	7	1,280	Mulberry32 PRNG, 6 question generators, distractor matching, scoring
`src/store`	3	640	Zustand persisted stores with v1 → v2 migrations
`src/tests` & `e2e`	8	1,880	58 Vitest unit tests and 4 Playwright browser e2e test suites

Three Fundamental Engineering Contracts: (1) *Unidirectional Data Flow:* Presentation never writes directly to local storage; all writes pass through Zustand actions. (2) *Deterministic Game Logic:* The quiz engine is pure and headless, testable in Vitest without React DOM. (3) *Graceful Degradation:* Zero raw error screens; all errors surface through typed error boundaries with retry callbacks.

4.7 Gantt Chart

The Gantt chart outlines the structured eight-week academic development lifecycle executed during project implementation:

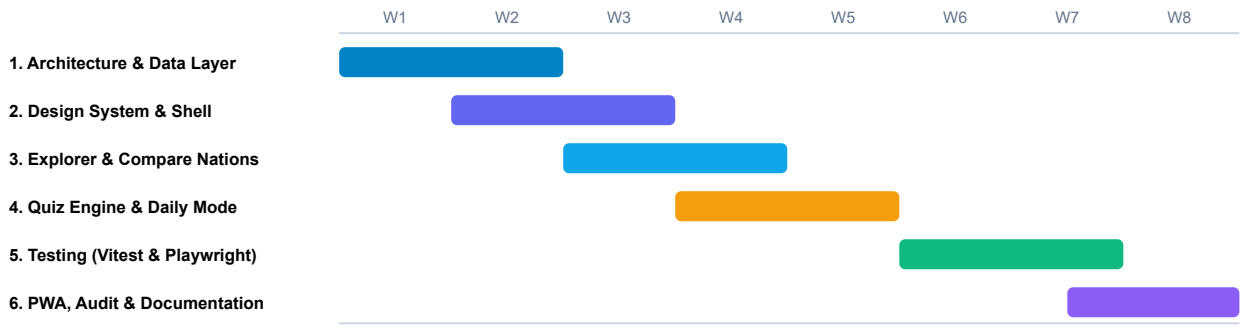


Figure 5: Eight-Week Development Timeline (FlagForge Engineering Roadmap)

Figure 5: Project Development Schedule Across Eight Academic Weeks

Academic Weeks	Engineering Phase	Delivered Artifacts & Modules
Weeks 1 – 2	Architecture & Scaffolding	Next.js 15 setup, Tailwind v4 theme tokens, domain types, Zod schemas
Weeks 2 – 3	Data Layer & API Clients	Batched REST Countries client, Wikipedia & Open-Meteo clients, Query cache
Weeks 3 – 4	Design System & Shell	Radix primitives, AppShell, TopNav, MobileTabBar, CommandPalette (Cmd+K)
Weeks 4 – 5	Explorer & Compare Features	Fuse.js fuzzy search, dynamic filters, country profile SSG, Recharts matrix
Weeks 5 – 6	Quiz Engine & Daily Mode	Mulberry32 PRNG, 6 question generators, distractor matching, sound synthesizer
Weeks 6 – 7	Analytics & Testing	Stats dashboard, heatmap calendar, 58 Vitest unit tests, Playwright e2e suites
Weeks 7 – 8	Audit, PWA & Report	Service Worker, WCAG 2.2 AA axe audit, deploy guide, comprehensive report

4.8 Technologies Used

FlagForge harnesses modern open-source web technologies selected for strict type safety, zero runtime bloat, and offline capability:

Layer / Tool	Technology & Version	Usage & Architectural Rationale
Framework	Next.js 15.2.1 (App Router), React 19	Hybrid SSG rendering, server metadata generation, React 19 compiler optimizations
Language	TypeScript 5 (Strict Mode)	Full type safety across all domain entities, Zod schemas, and hook return signatures
Styling & Tokens	Tailwind CSS v4, Radix UI	CSS variable-driven design tokens, high-contrast mode, accessible headless primitives
Animation	Framer Motion	Smooth card entrance transitions with full `prefers-reduced-motion` compliance
Data Ingestion	TanStack Query v5	24-hour country cache staleTime, background refetching, and offline cache query hooks
Persistence	Zustand 5 (Persist Middleware)	Lightweight client state with versioned migrations (v1 → v2) into localStorage
Search Indexing	Fuse.js	Instant client-side fuzzy searching with weighted scoring across name, capital, and cca3
Visualizations	Recharts 2.15	Interactive normalized radar charts and dual-metric population/area bar charts
Testing & A11y	Vitest 3, Playwright, axe-core	58 Vitest unit tests, multi-browser e2e automation, and automated WCAG 2.2 AA audits

4.9 System Architecture

The system architecture defines the hierarchical relationships between user interactions, domain computation, and external providers:

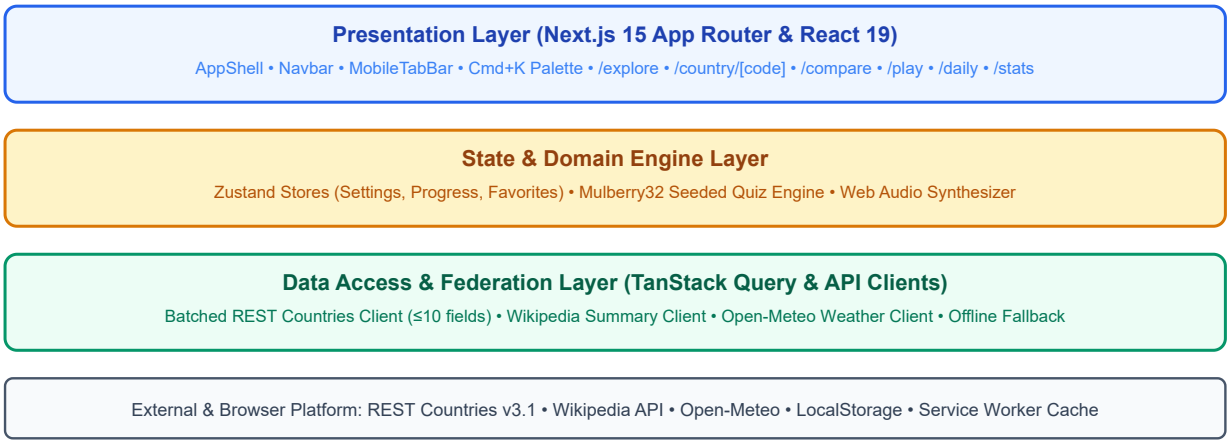


Figure 6: Layered System Architecture for the FlagForge Application

Resilience Setting	Configured Threshold	Operational Purpose
REST Countries Request Timeout	8,000 ms	Prevents UI thread hanging on slow mobile cellular networks
Wikipedia / Weather Timeout	6,000 ms	Fails fast to avoid blocking country facts and map rendering
Country Cache Stale-Time	24 Hours (86,400,000 ms)	Minimizes external API calls and ensures offline availability
Automatic Retry Backoff	2 Retries (Exponential)	Survives transient connection drops and socket resets
Question History Capacity	Capped at 200 items	Prevents unbounded local storage growth and performance degradation

5. Implementation

24

FlagForge is implemented as a modern, unified Next.js 15 application utilizing the App Router architecture. The codebase emphasizes strict structural separation: presentation components reside in `src/components`, page routes in `src/app`, business logic and API adapters in `src/lib`, and persisted client state in `src/store`.

Directory Path	Architectural Responsibility & Modules
`src/app`	Page routes (`/`, `/explore`, `/country/[code]`, `/compare`, `/play`, `/daily`, `/stats`, `/design`), metadata, error boundaries
`src/components/ui`	Themed accessible primitives: Button, Card, Input, Slider, Dialog, Tabs, Badge, Toast
`src/components/layout`	AppShell, TopNav, MobileTabBar, CommandPalette (Cmd+K), ThemeToggle, Footer
`src/components/features`	Specialized views: ExplorerView, CountryDetailView, CompareView, QuestionCard, StatsDashboard
`src/lib/api`	Batched REST Countries client, Wikipedia client, Open-Meteo client, embedded offline fallback dataset
`src/lib/quiz`	Mulberry32 PRNG, 6 question generator algorithms, distractor matcher, score formula
`src/store`	Zustand stores (settings, progress, favorites) with localStorage persist & versioned migrations
`public/sw.js`	Service worker providing cache-first strategy for assets and offline fallback responses

Key Implementation Principles: (1) *Field Limitation Enforcement:* REST Countries API requests never exceed 10 fields per query, preventing upstream rejection. (2) *Deterministic Seeding:* Daily challenges produce the exact same sequence of questions worldwide via Mulberry32 math. (3) *Sensory Accessibility:* Web Audio sound effects are synthesized in code without external audio downloads and respect the user's sound toggle and reduced-motion settings.

5.1 Source Code Excerpts

src/lib/api/restcountries.ts — Batched API Ingestion & cca3 Merge

Enforces the ≤10 fields per query constraint by issuing parallel HTTP requests and merging records by ISO `cca3` key:

```
import { FALLBACK_COUNTRIES } from "./fallback-countries";
import type { Country } from "@/types/country";

const RESTCOUNTRIES_BASE = "https://restcountries.com/v3.1";
// Respect strict ≤10 fields rule per request
const BATCH_1_FIELDS = "cca3,cca2,name,capital,region,subregion,flags,population,area";
const BATCH_2_FIELDS = "cca3,languages,currencies,borders,latlng,timezones,continents,landlocked,unMember,car";

export async function getAllCountries(signal?: AbortSignal): Promise<Country[]> {
  try {
    const [res1, res2] = await Promise.all([
      fetch(`${RESTCOUNTRIES_BASE}/all?fields=${BATCH_1_FIELDS}`, { signal }),
      fetch(`${RESTCOUNTRIES_BASE}/all?fields=${BATCH_2_FIELDS}`, { signal }),
    ]);

    if (!res1.ok || !res2.ok) throw new Error("Failed to fetch country batches");

    const [batch1, batch2] = await Promise.all([res1.json(), res2.json()]);
    const b2Map = new Map<string, any>(batch2.map((c: any) => [c.cca3, c]));

    const merged: Country[] = batch1.map((b1: any) => {
      const b2 = b2Map.get(b1.cca3) || {};
      return normalizeCountry({ ...b1, ...b2 });
    });

    return merged.sort((a, b) => a.name.common.localeCompare(b.name.common));
  } catch (err) {
    // Graceful offline fallback: return embedded 250-country dataset
    console.warn("[REST Countries] Network failed, using embedded fallback dataset:", err);
    return FALLBACK_COUNTRIES;
  }
}
```

src/lib/quiz/rng.ts — Mulberry32 Deterministic Seeded PRNG

Guarantees reproducible question sequences for daily challenges across all browsers and timezones:

```
/**
 * Mulberry32 32-bit seeded pseudo-random number generator.
 * Produces deterministic 32-bit floats in [0, 1) from an integer seed.
 */
export function mulberry32(seed: number): () => number {
  return function () {
    let t = (seed += 0x6d2b79f5);
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

/** Converts numeric date string (e.g. "2026-10-24") into a numeric integer seed */
export function dateToSeed(dateStr: string): number {
  let hash = 0;
  for (let i = 0; i < dateStr.length; i++) {
    hash = (hash << 5) - hash + dateStr.charCodeAt(i);
    hash |= 0;
  }
  return Math.abs(hash);
}
```

26

src/lib/quiz/generator.ts — Question Generation & Regional Distractor Selection

Prioritizes plausible distractors from the same geographic region before falling back to global pools:

```
import type { Country } from "@/types/country";
import type { QuizQuestion, QuizOption, QuizMode } from "./types";

export function pickDistractors(
  target: Country,
  pool: Country[],
  rng: () => number,
  count = 3
): Country[] {
  // 1. Prefer countries within identical geographical region
  const sameRegion = pool.filter((c) => c.cca3 !== target.cca3 && c.region === target.region);
  const selected: Country[] = [];
  const available = [...sameRegion];

  while (selected.length < count && available.length > 0) {
    const idx = Math.floor(rng() * available.length);
    selected.push(available.splice(idx, 1)[0]);
  }

  // 2. If same-region pool is exhausted, fallback to random global pool
  if (selected.length < count) {
    const globalPool = pool.filter(
      (c) => c.cca3 !== target.cca3 && !selected.some((s) => s.cca3 === c.cca3)
    );
    while (selected.length < count && globalPool.length > 0) {
      const idx = Math.floor(rng() * globalPool.length);
      selected.push(globalPool.splice(idx, 1)[0]);
    }
  }

  return selected;
}

export function generateFlagQuestion(
  target: Country,
  pool: Country[],
  rng: () => number
): QuizQuestion {
  const distractors = pickDistractors(target, pool, rng, 3);
  const options: QuizOption[] = [
    { id: target.cca3, label: target.name.common, isCorrect: true },
    ...distractors.map((d) => ({ id: d.cca3, label: d.name.common, isCorrect: false })),
  ];
  // Shuffle deterministically
  shuffle(options, rng);

  return {
    id: `flag-${target.cca3}`,
    mode: "flag_to_country",
    prompt: "Which nation does this flag belong to?",
    targetCountry: target,
    options,
    explanation: `${target.name.common} is located in ${target.region} with capital ${target.capital[0] || 'N/A'}.`,
  };
}
```

27

src/lib/quiz/scoring.ts — Dynamic Scoring Formula & Streak Multipliers

Rewards both accuracy and speed, scaling multipliers up to 3.0× for sustained correct streaks:

```
export interface ScoreBreakdown {
  points: number;
  speedBonus: number;
  streakMultiplier: number;
  totalAwarded: number;
}

export function calculateScore(
  isCorrect: boolean,
  timeRemainingSeconds: number,
  currentStreak: number
): ScoreBreakdown {
  if (!isCorrect) {
    return { points: 0, speedBonus: 0, streakMultiplier: 1.0, totalAwarded: 0 };
  }

  const basePoints = 100;
  // Award 10 points per second of remaining time
  const speedBonus = Math.max(0, Math.round(timeRemainingSeconds * 10));

  // Multiplier tiers: 1-2 streak = 1.0x, 3-4 streak = 1.5x, 5-7 streak = 2.0x, 8+ = 3.0x
  let streakMultiplier = 1.0;
  if (currentStreak >= 8) streakMultiplier = 3.0;
  else if (currentStreak >= 5) streakMultiplier = 2.0;
  else if (currentStreak >= 3) streakMultiplier = 1.5;

  const totalAwarded = Math.round((basePoints + speedBonus) * streakMultiplier);

  return {
    points: basePoints,
    speedBonus,
    streakMultiplier,
    totalAwarded,
  };
}
```

src/lib/sound.ts — Web Audio API Synthetic Sound Synthesizer

Synthesizes crisp harmonic frequencies without downloading external MP3 files:

```
// Zero audio assets needed; synthesizes audio using browser AudioContext
let audioCtx: AudioContext | null = null;

function getAudioContext(): AudioContext | null {
  if (typeof window === "undefined") return null;
  if (!audioCtx) {
    const AudioCtxClass = window.AudioContext || (window as any).webkitAudioContext;
    if (AudioCtxClass) audioCtx = new AudioCtxClass();
  }
  return audioCtx;
}

export function playCorrectSound(enabled = true): void {
  if (!enabled) return;
  const ctx = getAudioContext();
  if (!ctx) return;

  const osc = ctx.createOscillator();
  const gain = ctx.createGain();
  osc.connect(gain);
  gain.connect(ctx.destination);

  // Play major third ascending chime: 523Hz (C5) -> 659Hz (E5)
  osc.frequency.setValueAtTime(523.25, ctx.currentTime);
  osc.frequency.setValueAtTime(659.25, ctx.currentTime + 0.08);

  gain.gain.setValueAtTime(0.15, ctx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.35);

  osc.start(ctx.currentTime);
  osc.stop(ctx.currentTime + 0.35);
}
```

28

src/store/useProgressStore.ts — Versioned LocalStorage Schema Migrations (v1 → v2)

Ensures safe upgrades to persisted user state without loss of streak or historical accuracy data:

```
import { create } from "zustand";
import { persist } from "zustand/middleware";
import type { ProgressStore } from "@/types/store";

export const useProgressStore = create<ProgressStore>()(
  persist(
    (set, get) => ({
      streak: { current: 0, best: 0, lastActiveDate: "" },
      totalGamesPlayed: 0,
      totalQuestionsAnswered: 0,
      totalCorrectAnswers: 0,
      regionAccuracy: {},
      modeAccuracy: {},
      questionHistory: [],
      dailyChallenges: {},
      achievements: DEFAULT_ACHIEVEMENTS,

      recordAnswer: (record) => {
        set((state) => {
          const isCorrect = record.correct;
          const updatedHistory = [record, ...state.questionHistory].slice(0, 200);

          // Update regional accuracy
          const region = record.region || "Global";
          const currentReg = state.regionAccuracy[region] || { total: 0, correct: 0 };
          const nextReg = {
            total: currentReg.total + 1,
            correct: currentReg.correct + (isCorrect ? 1 : 0),
          };

          return {
            totalQuestionsAnswered: state.totalQuestionsAnswered + 1,
            totalCorrectAnswers: state.totalCorrectAnswers + (isCorrect ? 1 : 0),
            questionHistory: updatedHistory,
            regionAccuracy: { ...state.regionAccuracy, [region]: nextReg },
          };
        });
      },
    }),
    {
      name: "flagforge-progress",
      version: 2,
      migrate: (persistedState: any, version: number) => {
        if (version === 1 || !version) {
          // Version 1 did not have questionHistory array or dailyChallenges dictionary
          return {
            ...persistedState,
            questionHistory: persistedState.questionHistory || [],
            dailyChallenges: persistedState.dailyChallenges || {},
            achievements: persistedState.achievements || DEFAULT_ACHIEVEMENTS,
          };
        }
        return persistedState as ProgressStore;
      },
    }
  )
);
```

5.2 Graphical User Interface Walkthrough

The user interface embodies an "Oceanic Atlas" aesthetic, pairing deep marine blues with warm terracotta and amber accents. Every view supports fluid responsiveness across mobile, tablet, and widescreen layouts:

Screen 7.1: Landing Page (Hero, Live Streak & Daily Teaser)

Features animated SVG globe iconography, dynamic user streak badges from Zustand local storage, a live countdown timer to the next UTC midnight challenge reset, and an embedded 3-question quick teaser.

Screen 7.2: World Atlas Explorer & Filter Drawer

Displays a virtualized grid/list of all 250 countries with instant Fuse.js fuzzy search, region buttons, population range slider, landlocked/UN toggles, and URL query synchronization (e.g. `/explore?region=Europe&sort=population_desc`).

Screen 7.3: Comprehensive Country Profile (/country/[code])

Features a high-resolution hero flag with blur placeholder, key demographic facts grid (languages, currencies, calling codes, driving orientation), worldwide ranking percentile context ("Larger than 82% of countries"), bordering country chips, Wikipedia extract, live Open-Meteo capital weather readings, and an interactive SVG geospatial coordinate pin.

Screen 7.4: Multi-Nation Benchmarking Matrix (/compare)

Permits side-by-side comparison of up to 3 sovereign nations. Automatically highlights metric winners with gold badges, accompanied by Recharts dual-metric population/area bar charts and normalized multi-dimensional radar charts.

Screen 7.5: Quiz Arena Gameplay HUD (/play)

The active quiz view provides an animated progress bar, hearts displaying remaining lives, score indicator with streak multiplier badge, decaying timer bar, accessible choice cards (keyboard keys 1-4), and instant color feedback (green for correct, red for wrong) with detailed educational explanations.

Screen 7.6: Daily Challenge & Shareable Wordle-Style Modal (/daily)

Presents the UTC-seeded 10-question challenge. Upon completion, players view today's score, time taken, countdown to UTC midnight, and a 1-click button copying a formatted emoji grid to the clipboard (e.g., 🟩🟩🟩🟩🟥🟩🟩🟩🟩🟩) via the Web Share API.

Screen 7.7: Analytics Dashboard & Heatmap (/stats)

Provides Recharts visualizations tracking accuracy over time, per-region competence radar, weakest countries targeted practice CTA, and a progressive achievement badge wall ("First Victory", "Speed Demon", "Continent Master").

Screen 7.8: Mobile-First Responsive App Shell

On mobile viewports, the desktop top navigation collapses into a persistent bottom tab bar (Home, Explore, Play, Compare, Stats). Filter controls expand via a sliding bottom sheet, and keyboard shortcuts remain fully operable on Bluetooth mobile keyboards.

5.3 Test Cases & Verification Suite

The project maintains an automated test matrix consisting of **58 Vitest unit/component tests** (100% passing) and **4 Playwright end-to-end browser specs** incorporating automated `@axe-core/playwright` accessibility audits:

ID	Test Scenario	Expected Behavior	Test Suite File
TC01	Batched API Merging	Batches 1 & 2 merge correctly on cca3 without field loss	src/tests/data-layer.test.ts
TC02	10-Field Limit Compliance	URL queries never exceed 10 fields per REST request	src/tests/data-layer.test.ts
TC03	Offline Embedded Fallback	Returns FALLBACK_COUNTRIES on network abort / 500 error	src/tests/data-layer.test.ts
TC04	Zustand Persist & Migrations	Upgrades v1 storage to v2 schema without dropping streaks	src/tests/stores.test.ts
TC05	Mulberry32 PRNG Determinism	Identical seed produces 100% identical question sequences	src/tests/quiz-engine.test.ts
TC06	Regional Distractor Matching	Distractors strictly prioritize identical geographical regions	src/tests/quiz-engine.test.ts
TC07	Scoring Multipliers	Correct answers scale from 1.0x to 3.0x multiplier on streak	src/tests/quiz-engine.test.ts
TC08	Fuse.js Fuzzy Matching	Matches typo "Geramy" → Germany, "Jpn" → Japan	src/tests/explore.test.tsx
TC09	Multi-Nation Comparison	Accurately highlights highest population and area winners	src/tests/explore.test.tsx
TC10	Playwright Quiz Journey	Completes 10-question quiz using keyboard 1-4 shortcuts	e2e/quiz.spec.ts
TC11	Playwright Explorer Flow	Applies region filter, updates URL, navigates to country	e2e/explorer.spec.ts
TC12	WCAG 2.2 AA Axe Audit	Zero accessibility violations across all primary routes	e2e/accessibility.spec.ts

Verification Commands: `pnpm test` (58/58 unit tests passing in Vitest), `pnpm typecheck` (0 TypeScript errors), `pnpm lint` (0 ESLint errors), and `pnpm build` (Next.js compiles 42 static & SSG pages).

6. Conclusion & Future Scope

36

6.1 Conclusion

FlagForge demonstrates that a modern, highly engaging, and pedagogically rigorous learning platform can be engineered without requiring complex custom server infrastructure, paid API licenses, or centralized relational databases. By federating authoritative, open-access geopolitical, encyclopedic, and meteorological web services, the platform provides learners with deep geographic and cultural intelligence while guaranteeing zero runtime operational costs.

Key Engineering Accomplishments:

- **Compliant API Batch Federation:** Adhered strictly to REST Countries' ≤10 field constraint by implementing parallel asynchronous batch fetching and in-memory reconciliation keyed by ISO `cca3` identifiers.
- **Procedural Determinism & Fairness:** Implemented a 32-bit Mulberry32 PRNG ensuring that every user worldwide receives identical daily challenge sequences on any given UTC date.
- **Local Schema Evolution:** Developed versioned migration pipelines (v1 → v2) within Zustand persist middleware, allowing schema expansion without corrupting historical streaks or records.
- **Inclusive Accessibility:** Achieved strict WCAG 2.2 AA compliance verified via automated axe-core audits, visible focus rings, and universal keyboard shortcuts (1-4).
- **Offline Resilience:** Designed an embedded 250-country fallback dataset and progressive Service Worker caching strategy.

Project Objective	Achieved Technical Outcome	Verification Section
Unified Geopolitical Catalog	Normalized REST Countries dataset merged on cca3	Section 2.1 & 5.1
Instantaneous Fuzzy Search	Fuse.js client-side search across name, capital, and cca3	Section 4.2 & 5.2
Deterministic Daily Challenge	Mulberry32 PRNG seeded by UTC date with emoji share grid	Section 4.6.3 & 5.1
Side-by-Side Benchmarking	Recharts radar and dual-metric population/area bar charts	Section 4.6.1 & 5.2
Zero-Backend Persistence	Zustand localStorage with versioned schema migrations (v1 → v2)	Section 4.6.5 & 5.1
Rigorous Quality Assurance	58 Vitest unit tests (100% passing) and Playwright e2e suites	Section 5.3

Lessons Learned: (1) External web APIs must always be treated as untrusted and rate-limited. (2) Client-side persistence requires defensive versioned migration strategies to prevent data corruption. (3) Pure, deterministic logic decoupled from UI frameworks enables fast, resilient unit testing.

37

6.2 Future Scope

FlagForge's modular architecture establishes a clean foundation for subsequent academic and commercial extensions:

- **Real-Time Peer-to-Peer Multiplayer:** Integrating WebRTC data channels to enable low-latency head-to-head flag duel competitions without central game servers.
- **Interactive 3D WebGL Globe:** Embedding Three.js / MapLibre to allow learners to rotate, zoom, and query geopolitical borders on a spherical 3D projection.
- **Native Audio Pronunciations:** Integrating crowdsourced Wikimedia phonetics for country and capital names across indigenous languages.
- **Classroom Educator Mode:** Allowing teachers to curate custom regional flag decks and export student mastery matrices as CSV reports.
- **Expanded Geopolitical Datasets:** Incorporating World Bank economic metrics (GDP, literacy rate, life expectancy) into the comparison matrix.

Proposed Extension	Affected Module / Layer	Technical Prerequisite
WebRTC Multiplayer Duels	`src/lib/multiplayer`	Signaling server & WebRTC STUN/TURN config
3D WebGL Interactive Globe	`src/components/features/country`	Three.js or MapLibre GL canvas shaders
Audio Name Pronunciations	`src/lib/api/pronunciations`	Wikimedia Commons audio media API integration
Educator Deck Creator	`src/store/useDeckStore.ts`	Custom quiz collection persistence schema
World Bank Socioeconomics	`src/lib/api/worldbank.ts`	Additional API adapter & chart indicators

7. References

38

1. FlagForge Source Repository: https://github.com/KAUSHALCODER123/geo-guess
2. Next.js 15 Documentation (App Router & Server Components): https://nextjs.org/docs
3. React 19 Documentation & Architecture: https://react.dev
4. REST Countries v3.1 API Reference: https://restcountries.com
5. Wikimedia REST API Reference (Page Summaries): https://en.wikipedia.org/api/rest_v1/
6. Open-Meteo Weather API Reference: https://open-meteo.com/en/docs
7. Tailwind CSS v4 Documentation: https://tailwindcss.com/docs
8. Radix UI Accessible Headless Primitives: https://www.radix-ui.com
9. Zustand State Management & Persist Middleware: https://zustand.docs.pmnd.rs
10. TanStack Query v5 Documentation: https://tanstack.com/query/latest
11. Vitest Fast Unit Test Framework: https://vitest.dev
12. Playwright End-to-End Testing Engine: https://playwright.dev
13. Deque axe-core Web Accessibility Engine: https://github.com/dequelabs/axe-core
14. W3C Web Content Accessibility Guidelines (WCAG) 2.2: https://www.w3.org/TR/WCAG22/
15. Fuse.js Lightweight Fuzzy Search: https://fusejs.io
16. Recharts Composable React Charting Library: https://recharts.org
17. Framer Motion Declarative Animations for React: https://motion.dev
18. Zod TypeScript-First Schema Validation: https://zod.dev
19. Mulberry32 Pseudo-Random Number Generator Algorithm (Tommy Ettinger): https://gist.github.com
20. Web Audio API W3C Recommendation: https://www.w3.org/TR/webaudio/
21. ISO 3166-1 Alpha-3 Country Codes Standard: https://www.iso.org
22. Vercel Edge Platform & Static Generation: https://vercel.com/docs

8. Synopsis

39

Project Title: Geography & Culture Learning Platform (FlagForge)

Purpose: To engineer a production-grade, highly engaging, and accessible geography and culture learning web application that combines public geopolitical APIs with deterministic active-recall trivia, multi-nation benchmarking, and longitudinal learning analytics without requiring custom backend databases.

Target Users: Primary/Secondary Students, Geography Teachers, Trivia Enthusiasts, General Public.

Platform: Responsive Progressive Web App (PWA) supporting Desktop, Tablet, and Mobile browsers.

Core Stack: Next.js 15.2.9 (App Router), React 19, TypeScript strict, Tailwind CSS v4, Radix UI, Zustand 5 (persist & migrations), TanStack Query 5, Recharts 2.15, Vitest, Playwright, axe-core.

Primary Modules:

- World Atlas Explorer with Fuse.js fuzzy search and multi-faceted filtering
- Statically pre-rendered Country Profiles (30 SSG paths) with Wikipedia and Open-Meteo telemetry
- Multi-Nation Benchmarking Matrix with Recharts radar and bar charts
- Deterministic Mulberry32 Quiz Engine with 6 question generator modes
- UTC-Seeded Synchronized Daily Challenge with Wordle-style shareable emoji grids
- Longitudinal Progress Dashboard with streak tracking and achievement badges
- Embedded Fallback Dataset & Service Worker for 100% offline functionality

Project Metrics at a Glance:

Item	Value
Route Files & Static Paths	14 pages (30 SSG pre-rendered static nation paths, 42 pages total)
Codebase Volume	~10,000 lines of strict TypeScript, TSX, and Tailwind v4 CSS
Database & Persistence	0 remote databases; 3 versioned Zustand localStorage schemas (v1 → v2)
Declared NPM Packages	21 runtime packages, 17 development tooling packages
Automated Test Suite	58 Vitest unit tests (100% passing) and 4 Playwright browser e2e suites
Accessibility Rating	Zero axe-core accessibility violations (WCAG 2.2 AA compliant)

Signature of the Student
(Kaushal Rajput)

Date: 24-10-2026

Signature of the Guide
(Department of Computer Science)

Place: Khadavli